Question 6: Loops That Must Terminate: Loan Repayment Schedule

Student name	**Sk.Thohith**
Register number	**1912260555**
GitHub repository	CSA02-<RegisterNumber>-Assignment2 (folder: Q6)
Link	https://github.com/shaikthohith5353-ship-it/CSA02-1912260555-Assignment-2
Compiler	gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Compile command	`gcc -Wall -Wextra -O2 -o q6_loan q6_loan.c -lm`
Marks / Bloom level	8 marks / K5 – Evaluate

1. Problem and loop design

A self-help group lends money at a monthly interest rate. Each month the interest is added to the outstanding balance and the member repays a fixed amount E. The program prints the month-by-month schedule until the loan is fully repaid and reports the number of months and the total interest paid. Notation: P = loan, r = monthly rate (r = 2% means 0.02), E = fixed monthly repayment.

1.1 Which loop?

Loop	Fit for this problem?	Reason
for	No	The number of months is the unknown result, not a known count. A for loop would hide the real stopping rule inside a fake bound.
while	**Yes (chosen)**	The rule is a condition (“money is still owed”). It is tested *before* each month, so an already-zero balance runs zero months and no negative payment is produced.
do-while	No	It always runs the body once, even when there is nothing to repay.

1.2 Termination condition and the floating-point trap

The loop continues while `balance > 0.005` (half a paisa). The test `balance != 0` is dangerous because a double cannot represent most decimal amounts exactly, so a balance that is mathematically zero is often a tiny non-zero value (or slightly negative). Experiments (file q6_experiments.c):

Experiment	Result	Meaning
P = 1.00, r = 0, E = 0.10, payment clamped, test **balance != 0**	11 months (should be 10)	After 10 payments balance = 1.4×10^{-16}, not 0; an extra month is needed
same, test **balance > 0.005**	10 months	correct
P = 1.00, E = 0.10, no clamp, test **balance != 0**	never stops (cap at 50 months, balance − 4.0)	balance jumps over 0 to a negative value
P = 10000, 2%, E = 2000, no clamp, test **balance != 0**	cap at 360 months, balance − 112,180,501.50	the loan is “overpaid” forever; the loop never ends

1.3 The last month and an inadequate repayment

- **Last month.** When the balance plus interest is smaller than E, the member pays only what is left: `payment = (repayment < balance) ? repayment : balance;`. Without this clamp the final balance would be negative (an overpayment) and the total paid would be wrong.
- **E smaller than the monthly interest.** The balance then *grows* every month and the loop would never end. If E equals the interest, the balance stays constant (interest-only) and it also never ends. The program therefore stops with a message when `repayment <= interest`; in the example below E = 200 = interest and the program stops after month 1.

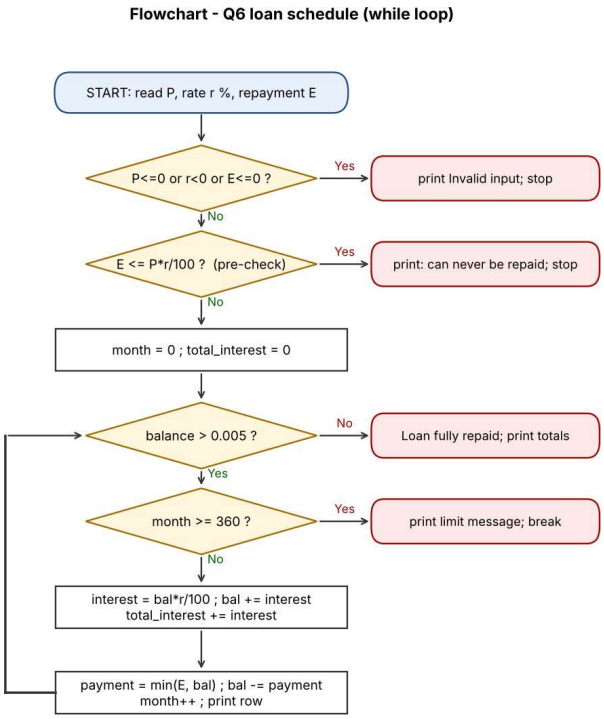


Figure 1: flowchart of the final design (while loop, pre-check, tolerance, 360-month limit)

2. Student program and execution

The program written by the student (screenshot below) uses the while loop, the 0.005 tolerance, the clamped last payment, the `repayment <= interest` test and the 360-month limit. Source in Appendix A.

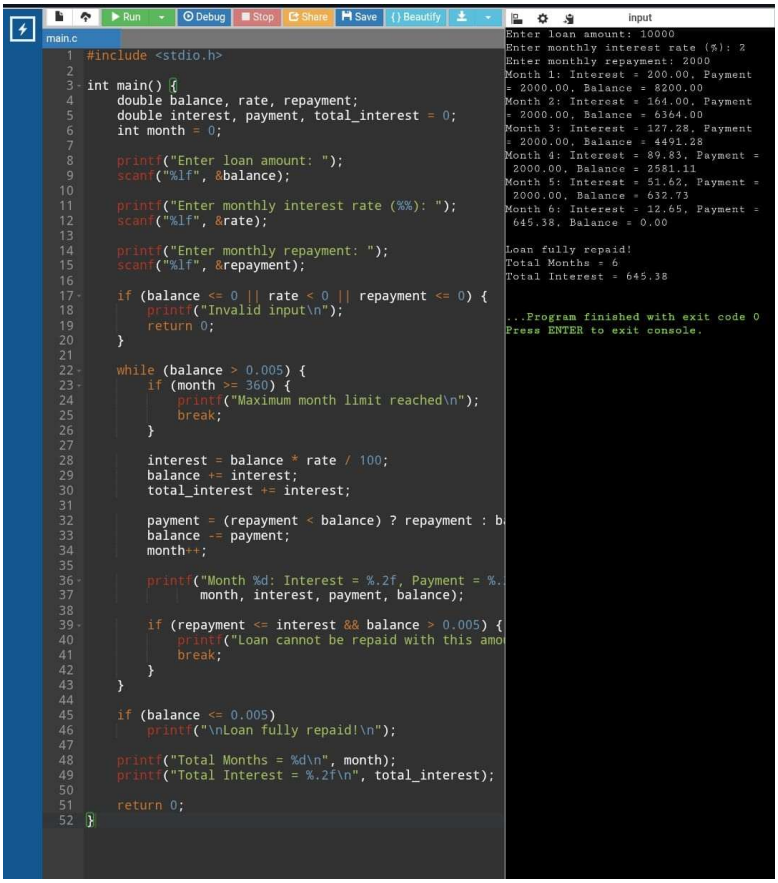


Figure 2: student program and its output for Rs 10,000, 2% per month, Rs 2,000 repayment

3. Correctness check: hand schedule versus program

Hand calculation for P = Rs 10,000, r = 2% per month, E = Rs 2,000 (interest = balance × 0.02; new balance = balance + interest − payment):

Month	Opening balance	Interest (hand)	Balance + interest	Payment	Closing balance (hand)	Program output
1	10,000.0000	200.0000	10,200.0000	2,000.00	8,200.0000	8200.00
2	8,200.0000	164.0000	8,364.0000	2,000.00	6,364.0000	6364.00
3	6,364.0000	127.2800	6,491.2800	2,000.00	4,491.2800	4491.28
4	4,491.2800	89.8256	4,581.1056	2,000.00	2,581.1056	2581.11
5	2,581.1056	51.6221	2,632.7277	2,000.00	632.7277	632.73
6	632.7277	12.6546	645.3823	**645.38** (last)	0.0000	0.00

Total interest by hand = 200 + 164 + 127.28 + 89.8256 + 51.6221 + 12.6546 = **645.3823**; months = **6**. Cross-check: total paid = 5 × 2,000 + 645.38 = 10,645.38 = loan + interest. The program printed **Total Months = 6** and **Total Interest = 645.38**: the hand schedule and the program agree in every row.

```
Enter loan amount: Enter monthly interest rate (%): Enter monthly repayment: Month 1: Interest = 200.00, Payment
     = 2000.00, Balance = 8200.00
Month 2: Interest = 164.00, Payment = 2000.00, Balance = 6364.00
Month 3: Interest = 127.28, Payment = 2000.00, Balance = 4491.28
Month 4: Interest = 89.83, Payment = 2000.00, Balance = 2581.11
Month 5: Interest = 51.62, Payment = 2000.00, Balance = 632.73
Month 6: Interest = 12.65, Payment = 645.38, Balance = 0.00

Loan fully repaid!
Total Months = 6
Total Interest = 645.38
```

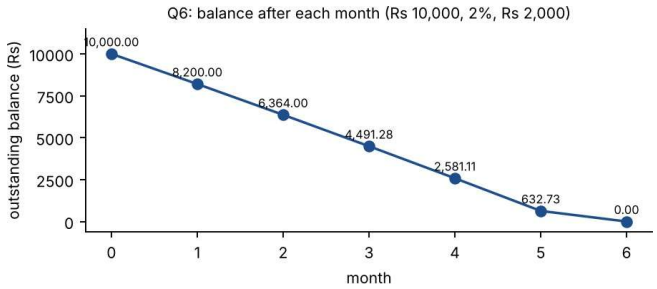


Figure 3: outstanding balance after each month

4. Growth investigation: repayment approaching the interest

Prediction. Solving the recurrence gives n = −ln(1 − rP/E) / ln(1 + r). With rP = 200 the argument 1 − 200/E tends to 0 as E falls towards 200, so n grows without bound (logarithmically in 1/(E − 200)). Predicted: moderate growth for large E, rapid growth near E = 200; every time E − 200 is divided by 5, about ln 5 / ln 1.02 ≈ 81 extra months. Measured (simulation, file q6_experiments.c):

E (Rs)	Months (loop iterations)	Predicted n (formula)	ceil(n)	Total interest (Rs)
5000	3	2.061	3	310.08
3000	4	3.484	4	459.50
2000	6	5.321	6	645.38
1500	8	7.226	8	842.14
1000	12	11.268	12	1,270.33
700	17	16.991	17	1,893.96
500	26	25.796	26	2,898.73
400	36	35.003	36	4,001.13
300	56	55.478	56	6,644.17
250	82	81.274	82	10,318.98
220	122	121.090	122	16,639.92
205	188	187.529	188	28,444.00
201	268	267.808	268	43,829.75
200.5	303	302.685	303	50,688.80
200.1	> 360 (limit hit, Rs 3,767.19 left)	383.858	384	65,803.19 at month 360
200	never (balance stays 10,000)	undefined	–	–
150	never (balance grows)	undefined	–	–

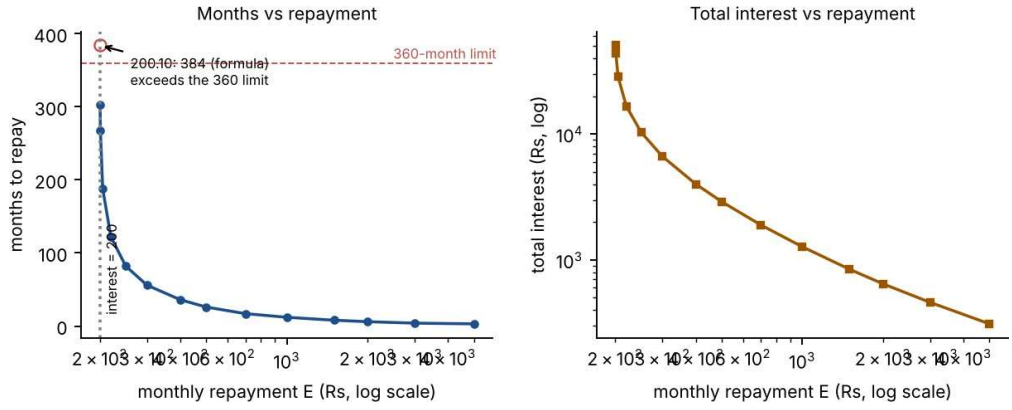


Figure 4: months and total interest against the repayment (measured)

Observations. Halving E from 2,000 to 1,000 doubles the months (6 → 12) and almost doubles the interest paid. Between 400 and 250 the months more than double again; between 205 and 200.5 the schedule stretches from 188 to 303 months, and at 200.1 it passes the 360-month limit. The measured steps agree with the prediction (E−200 = 5 → 1: +80 months, predicted 81). The ceil of the formula equals the simulated number of months for every row.

5. Deployment decision: safety limit and messages

The limit of **360 months** (30 years) is implemented with break. Recommended final behaviour (file q6_loan_final.c, same schedule output as above):

Situation	Message / action	Reason
Invalid input (P ≤ 0, r < 0, E ≤ 0)	"Invalid input", exit	protects the arithmetic
E ≤ first month's interest	"Repayment Rs X does not exceed the first month's interest Rs Y: loan can never be repaid." (checked **before** the loop)	detects the impossible loan at once instead of printing a misleading month 1 or waiting 360 iterations
Still unpaid after 360 months	"Not repaid within 360 months. Outstanding balance Rs B. Increase the repayment."; the totals are still printed; "Loan fully repaid" is **not** printed	safety net for borderline loans (E = 200.1 needs 384 months) and for any rounding or logic error

```
### Input: P=10000 rate=2% E=2000
Month 2: Interest = 164.00, Payment = 2000.00, Balance = 6364.00
Month 3: Interest = 127.28, Payment = 2000.00, Balance = 4491.28
Month 4: Interest = 89.83, Payment = 2000.00, Balance = 2581.11
Month 5: Interest = 51.62, Payment = 2000.00, Balance = 632.73
Month 6: Interest = 12.65, Payment = 645.38, Balance = 0.00

Loan fully repaid!
Total Months = 6
Total Interest = 645.38

### Input: P=10000 rate=2% E=200
Enter loan amount: Enter monthly interest rate (%): Enter monthly repayment: Repayment Rs 200.00 does not exceed
     the first month's interest Rs 200.00: loan can never be repaid.

### Input: P=10000 rate=2% E=150
Enter loan amount: Enter monthly interest rate (%): Enter monthly repayment: Repayment Rs 150.00 does not exceed
     the first month's interest Rs 200.00: loan can never be repaid.

### Input: P=10000 rate=2% E=200.1
Month 360: Interest = 77.79, Payment = 200.10, Balance = 3767.19
Not repaid within 360 months. Outstanding balance Rs 3767.19. Increase the repayment.
Total Months = 360
Total Interest = 65803.19

### Input: P=5000 rate=0% E=1000
Month 2: Interest = 0.00, Payment = 1000.00, Balance = 3000.00
Month 3: Interest = 0.00, Payment = 1000.00, Balance = 2000.00
Month 4: Interest = 0.00, Payment = 1000.00, Balance = 1000.00
Month 5: Interest = 0.00, Payment = 1000.00, Balance = 0.00

Loan fully repaid!
Total Months = 5
Total Interest = 0.00

### Input: P=500 rate=2% E=1000
Enter loan amount: Enter monthly interest rate (%): Enter monthly repayment: Month 1: Interest = 10.00, Payment
    = 510.00, Balance = 0.00

Loan fully repaid!
Total Months = 1
Total Interest = 10.00

### Input: P=0 rate=2% E=1000
Enter loan amount: Enter monthly interest rate (%): Enter monthly repayment: Invalid input
```

Defence. (1) The pre-check gives the member a precise reason and the exact minimum needed (more than Rs 200); the 360-month limit alone would only say "something is wrong" after printing hundreds of lines. (2) The limit is still kept because it is the only guard against loops whose end depends on floating-point behaviour and against loans that are technically repayable but beyond any

realistic horizon. (3) 360 months is long compared with micro-loans (typically a few months to a few years) and is measured: the E = 200.1 case shows it triggers with Rs 3,767.19 outstanding. (4) The message states the outstanding balance and the remedy, and the program never claims the loan is repaid when it is not.

6. Alternative concept: direct formula

How the solution changes. The number of months is computed directly: n = −log(1 − rP/E) / log(1 + r), and the months are ceil(n). No loop and no schedule are produced. The total interest needs the last payment: after k = months − 1 full payments the balance is $B_k = P(1+r)^k - E((1+r)^k - 1)/r$, the last payment is $B_k(1+r)$, and total interest = $k \cdot E + B_k(1+r) - P$.

```
double x = r * P / E;                    /* r as a fraction, e.g. 0.02 */
if (x >= 1.0) { puts("never repaid"); }
else {
    double n = -log1p(-x) / log1p(r);     /* stable form of -log(1-x)/log(1+r) */
    int months = (int)ceil(n - 1e-9);     /* guard against 6.0000000001 */
}
```

Aspect	Simulation loop	Closed formula
Time	O(n) iterations, n = months (3 … 303 measured; 360 max)	O(1): two logarithms
Space	O(1)	O(1)
Output	full schedule, exact last payment, total interest by summation	months only (schedule needs the loop anyway)
Precision	each month adds rounding error (about n·ε in total); needs a tolerance such as 0.005	ceil() is fragile at exact integers: the exact 6-month payment E = 1785.2581233520 gives n = 5.999999999999999 (ceil = 6, correct) but a value like 6.0000000000000009 would give 7; needs a small epsilon
Edge cases	r = 0 works (5,000 ÷ 1,000 = 5 months); E ≤ rP detected by comparison	r = 0 gives log(1) = 0 and 0/0 = NaN (measured); E ≤ rP makes the logarithm undefined and must be tested first

Measured agreement: for all 14 loans that terminate within 360 months, ceil(formula) equals the simulated months (table in section 4). When 1 − rP/E is very small, the plain form `log(1 - x)` loses digits (E = 200.0001: 1 − x = 5.0×10^{-7}), so `log1p(-x)` is preferred; for these values both forms still printed 732.663299, so the loss stayed below 10^{-6} in double precision.

Recommendation. Use the loop to produce the schedule (the assignment requires it), and use the formula as a O(1) feasibility check and an independent cross-check of the number of months. Neither replaces the other.

7. Test table

Input (P, r %, E)	Purpose	Expected	Observed
10000, 2, 2000	spec example, hand schedule	6 months, interest 645.38	6 months, 645.38
10000, 2, 200	E = interest	refused, cannot be repaid	message printed, no endless loop
10000, 2, 150	E < interest	refused	message printed
10000, 2, 200.1	just above interest, long loan	limit message at 360	limit message, Rs 3,767.19 left
5000, 0, 1000	zero interest	5 months, interest 0	5 months, 0.00
500, 2, 1000	E larger than balance + interest	1 month, payment 510.00	1 month, payment 510.00
0, 2, 1000	invalid	Invalid input	Invalid input

8. Conclusion

A while loop with a tolerance-based condition, a clamped last payment, a check for repayments that cannot cover the interest and a 360-month limit terminates for every input tested. The hand schedule matches the program in every month; the iteration counts grow slowly and then explosively as E approaches the interest, which is the reason both the pre-check and the limit are justified.

Appendix A: source code (executed with gcc 13.3.0)

q6_loan.c – student program (while loop, tolerance, limit)

```
#include <stdio.h>
#include <math.h>

/* Loan schedule: while loop, tolerance test, clamped last payment, 360-month safety limit */
int main(void) {
    double balance, rate, repayment;
    double interest, payment, total_interest = 0;
    int month = 0;

    printf("Enter loan amount: ");             scanf("%lf", &balance);
    printf("Enter monthly interest rate (%%): "); scanf("%lf", &rate);
    printf("Enter monthly repayment: ");  scanf("%lf", &repayment);

    if (balance <= 0 || rate < 0 || repayment <= 0) { printf("Invalid input\n"); return 0; }

    while (balance > 0.005) {
        if(month >= 360) { printf("Maximum month limit reached\n"); break; }
        interest = balance * rate / 100;
        balance   +=   interest;
        total_interest += interest;
        payment = (repayment < balance) ? repayment : balance;
        balance -= payment;
        month++;
        printf("Month %d: Interest = %.2f, Payment = %.2f, Balance = %.2f\n",
               month, interest, payment, balance);
        if (repayment <= interest && balance > 0.005) {
            printf("Loan cannot be repaid with this amount\n");
            break;
        }
    }
    if (balance <= 0.005) printf("\nLoan fully repaid!\n");
    printf("Total Months = %d\n", month);
    printf("Total Interest = %.2f\n", total_interest);
    return 0;
}
```

q6_loan_final.c – recommended deployment version

```c
#include <stdio.h>
#define MAX_MONTHS 360    /* safety limit: 30 years */

int main(void) {
    double balance, rate, repayment, interest, payment, total_interest = 0.0;
    int month = 0;

    printf("Enter loan amount: ");
    if (scanf("%lf", &balance) != 1) return 1;
    printf("Enter monthly interest rate (%%): ");
    if (scanf("%lf", &rate) != 1) return 1;
    printf("Enter monthly repayment: ");
    if (scanf("%lf", &repayment) != 1) return 1;

    if (balance <= 0 || rate < 0 || repayment <= 0) { printf("Invalid input\n"); return 0; }

    /* Pre-check: if the repayment cannot even cover the first month's interest, the balance never falls */
    interest = balance * rate / 100.0;
    if (repayment <= interest) {
        printf("Repayment Rs %.2f does not exceed the first month's interest Rs %.2f: loan can never be
            repaid.\n",
               repayment, interest);
        return 0;
    }

    while (balance > 0.005) {                  /* tolerance: half a paisa */
        if(month >= MAX_MONTHS) {
            printf("Not repaid within %d months. Outstanding balance Rs %.2f. Increase the repayment.\n",
                   MAX_MONTHS, balance);
            break;
        }
        interest = balance * rate / 100.0;
        balance   +=   interest;
        total_interest += interest;
        payment = (repayment < balance) ? repayment : balance;/* last month: pay only what is left */
        balance -= payment;
        month++;
        printf("Month %d: Interest = %.2f, Payment = %.2f, Balance = %.2f\n", month, interest, payment,
            balance);
    }
    if (balance <= 0.005) printf("\nLoan fully repaid!\n");
    printf("Total Months = %d\n", month);
    printf("Total Interest = %.2f\n", total_interest);
    return 0;
}
```

q6_experiments.c – growth, floating-point and formula experiments

```c
#include <stdio.h>
#include <math.h>

/* returns months, or -1 if limit reached; iterations = months */
static int simulate(double P, double r_pct, double E, double *tot_int, double *final_bal){
    double bal = P, ti = 0; int m = 0;
    while (bal > 0.005) {
        if (m >= 360) { *tot_int = ti; *final_bal = bal; return -1; }
        double in = bal * r_pct / 100; bal += in; ti += in;
        double pay = (E < bal) ? E : bal; bal -= pay; m++;
    }
    *tot_int = ti; *final_bal = bal; return m;
}
int main(void){
    double P = 10000, r = 2.0;
    printf("== Growth investigation: P=10000, r=2%% (interest in month 1 = 200) ==\n");
    printf("%10s %10s %12s %14s %14s\n","repayment","months","formula n","ceil(formula)","total interest");
    double E[] = {5000,3000,2000,1500,1000,700,500,400,300,250,220,205,201,200.5,200.1,200,150};
    int nE = sizeof E / sizeof E[0];
    for (int k = 0; k < nE; k++){
        double ti, fb; int m = simulate(P, r, E[k], &ti, &fb);
        double rr = r/100.0, arg = 1.0 - rr*P/E[k];
        char fs[32] = "n/a", cs[32] = "n/a";
        if (arg > 0) { double n = -log(arg)/log(1+rr); snprintf(fs,32,"%.3f",n); snprintf(cs,32,"%.0f",ceil(n));
            }
        if (m < 0) printf("%10.2f %10s %12s %14s(limit; balance left %.2f)\n", E[k], ">360", fs, cs, fb);
        else printf("%10.2f %10d %12s %14s %14.2f\n", E[k], m, fs, cs, ti);
    }
    /* exact hand-check numbers */
    printf("\n== Floating-point danger: balance != 0 ==\n");
    {   /* rate 0, P=1.0, E=0.1 : should take exactly 10 months */
        double bal = 1.0; int m = 0;
        while (bal != 0 && m < 50) { double pay = (0.1 < bal) ? 0.1 : bal; bal -= pay; m++; }
        printf("Clamped payment, test 'balance != 0': months=%d (expected 10), balance=%.3e\n", m, bal);
        bal = 1.0; m = 0;
        while (bal > 0.005 && m < 50) { double pay = (0.1 < bal) ? 0.1 : bal; bal -= pay; m++; }
        printf("Clamped payment, test 'balance > 0.005': months=%d\n", m);
        bal = 1.0; m = 0;
        while (bal != 0 && m < 50) { bal -= 0.1; m++; }
        printf("No clamp, test 'balance != 0': stopped by cap at months=%d, balance=%.3e (never exactly 0)\n",
            m, bal);
        bal = 10000; m = 0;
        while (bal != 0 && m < 360) { bal += bal*0.02; bal -= 2000; m++; }
        printf("Real loan, no clamp, 'balance != 0': cap hit at m=%d, balance=%.2f (negative: overpaid, never
            0)\n", m, bal);
    }
    printf("\n== Formula vs simulation precision ==\n");
    {
        double rr = 0.02;
        double Evs[] = {200.0001, 200.01, 200.4};
        for (int q = 0; q < 3; q++) {
            double Ev = Evs[q];
            double x = rr*P/Ev;
            double n_naive = -log(1.0 - x)/log(1.0 + rr);
            double n_stable = -log1p(-x)/log1p(rr);
            printf("E=%.4f: naive formula n=%.6f, log1p form n=%.6f, 1-x=%.3e\n", Ev, n_naive, n_stable, 1.0-x);
        }
        /* integer-valued n : choose E so n is exactly 6? search E producing n close to integer */
        double Ev6 = rr*P / (1 - pow(1+rr, -6.0));  /* payment that clears loan in exactly 6 months */
        double n6 = -log(1.0 - rr*P/Ev6)/log(1.0+rr);
        double ti, fb; int m = simulate(P, 2.0, Ev6, &ti, &fb);
        printf("E=%.10f (exact 6-month annuity): formula n=%.15f, ceil=%.0f, simulation months=%d, final
            bal=%.3e\n", Ev6, n6, ceil(n6), m, fb);
        printf("E=%.2f (rounded to paise): formula n=%.6f, simulation months=%d\n", round(Ev6*100)/100, -log(1.0
             - rr*P/(round(Ev6*100)/100))/log(1.0+rr), simulate(P,2.0,round(Ev6*100)/100,&ti,&fb));
        printf("r=0: formula divides by log(1)=%.1f -> n = %f (inf/nan); simulation handles it.\n", log(1.0),
            -log(1.0-0.0)/log(1.0+0.0));
    }
    return 0;
}
```

Output of q6_experiments

```
== Growth investigation: P=10000, r=2% (interest in month 1 = 200) ==
  repayment      months    formula n   ceil(formula) total interest
    5000.00           3        2.061               3          310.08
    3000.00           4        3.484               4          459.50
    2000.00           6        5.321               6          645.38
    1500.00           8        7.226               8          842.14
    1000.00          12       11.268              12         1270.33
     700.00          17       16.991              17         1893.96
     500.00          26       25.796              26         2898.73
     400.00          36       35.003              36         4001.13
     300.00          56       55.478              56         6644.17
     250.00          82       81.274              82        10318.98
     220.00         122      121.090             122        16639.92
     205.00         188      187.529             188        28444.00
     201.00         268      267.808             268        43829.75
     200.50         303      302.685             303        50688.80
     200.10        >360      383.858             384   (limit; balance left 3767.19)
     200.00        >360          n/a             n/a   (limit; balance left 10000.00)
     150.00        >360          n/a             n/a   (limit; balance left 3126402.82)

== Floating-point danger: balance != 0 ==
Clamped payment, test 'balance != 0': months=11 (expected 10), balance=0.000e+00
Clamped payment, test 'balance > 0.005': months=10
No clamp, test 'balance != 0': stopped by cap at months=50, balance=-4.000e+00 (never exactly 0)
Real loan, no clamp, 'balance != 0': cap hit at m=360, balance=-112180501.50 (negative: overpaid, never 0)

== Formula vs simulation precision ==
E=200.0001: naive formula n=732.663299, log1p form n=732.663299, 1-x=5.000e-07
E=200.0100: naive formula n=500.112303, log1p form n=500.112303, 1-x=5.000e-05
E=200.4000: naive formula n=313.928349, log1p form n=313.928349, 1-x=1.996e-03
E=1785.2581233520 (exact 6-month annuity): formula n=5.999999999999999, ceil=6, simulation months=6, final
    bal=7.503e-12
E=1785.26 (rounded to paise): formula n=5.999993, simulation months=6
r=0: formula divides by log(1)=0.0 -> n = -nan (inf/nan); simulation handles it.
```